Отчёт по лабораторной работе №5

Математические основы защиты информации и информационной безопасности

Сунь Маосин

Содержание

1 Цель работы **5**

2 Реализация алгоритмов **6**

2.1 Вспомогательная функция быстрого возведения в степень 6

2.1.1 Код функции 7

2.2 Тест Ферма 7

2.2.1 Код реализации 8

2.3 Алгоритм вычисления символа Якоби 8

2.3.1 Код реализации 10

2.4 Тест Соловэя-Штрассена 11

2.4.1 Код реализации 13

2.5 Тест Миллера-Рабина 14

2.5.1 Код реализации 15

3 Тестирование алгоритмов **17**

3.0.1 Результаты тестирования 18

4 Вывод **20**

Список иллюстраций

2.1 Функция быстрого возведения в степень 7
2.2 Тест Ферма 8
2.3 Алгоритм вычисления символа Якоби 10
2.4 Тест Соловэя-Штрассена 13
2.5 Тест Миллера-Рабина (часть 1) 15
2.6 Тест Миллера-Рабина (часть 2) 16

3.1 Результаты тестирования 18

Список таблиц

1 Цель работы

Изучить и освоить вероятностные алгоритмы проверки чисел на простоту, широко используемые в криптографии. Реализовать программно тесты Ферма, Соловэя-Штрассена и Миллера-Рабина, а также понять их основные принципы и вероятность ошибки.

2 Реализация алгоритмов

2.1 Вспомогательная функция быстрого возведения в степень

Для эффективного вычисления больших степеней по модулю была реализована функция `mod_pow`, использующая бинарный метод возведения в степень.

2.1.1 Код функции

```
def mod_pow(base, exponent, mod):
    """
    Fast modular exponentiation
    Computes (base^exponent) % mod efficiently
    """
    result = 1
    base = base % mod

    while exponent > 0:
        # If exponent is odd, multiply result by base
        if exponent % 2 == 1:
            result = (result * base) % mod

        # Square the base and halve the exponent
        base = (base * base) % mod
        exponent = exponent // 2

    return result
```

Рис. 2.1: Функция быстрого возведения в степень

2.2 Тест Ферма

Тест Ферма основан на малой теореме Ферма: для простого числа p выполняется $a^{p-1} \equiv 1 \pmod{p}$ для любого a, не кратного p. Если для выбранного случайного a это условие не выполняется, число гарантированно составное.

2.2.1 Код реализации

```python
def fermat_test(n, k=5):
    """
    Fermat primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Perform k rounds of testing
    for _ in range(k):
        # Choose random a in [2, n-2]
        a = random.randint(2, n - 2)

        # Fermat's little theorem: a^(n-1) ≡ 1 (mod n) for primes
        if mod_pow(a, n - 1, n) != 1:
            return False

    # Passed all tests
    return True
```

Рис. 2.2: Тест Ферма

2.3 Алгоритм вычисления символа Якоби

Символ Якоби является обобщением символа Лежандра и необходим для теста Соловэя-Штрассена. Алгоритм использует свойства квадратичной взаимности для рекурсивного вычисления.

2.3.1 Код реализации

```
def jacobi_symbol(a, n):
    """
    Compute the Jacobi symbol (a/n)
    a: integer
    n: positive odd integer
    Returns: -1, 0, or 1
    """
    # Step 1: Initialization
    if a == 0:
        return 0
    if a == 1:
        return 1

    result = 1
    a = a % n

    # Step 2: Remove factors of 2 from a
    while a % 2 == 0:
        a = a // 2
        # Check quadratic reciprocity for factor 2
        if n % 8 == 3 or n % 8 == 5:
            result = -result

    # Step 3: Apply quadratic reciprocity
    if a == 1:
        return result

    # If both are 3 mod 4, flip sign
    if a % 4 == 3 and n % 4 == 3:
        result = -result

    # Step 4: Swap a and n and continue
    return result * jacobi_symbol(n % a, a)
```

Рис. 2.3: Алгоритм вычисления символа Якоби

2.4 Тест Соловэя-Штрассена

Тест Соловэя-Штрассена основан на критерии Эйлера: для простого числа p выполняется $a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}$, где $\left(\frac{a}{p}\right)$ — символ Лежандра. В тесте используется символ Якоби для проверки этого условия.

2.4.1 Код реализации

```
def solovay_strassen_test(n, k=5):
    """
    Solovay-Strassen primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Perform k rounds of testing
    for _ in range(k):
        # Choose random a in [2, n-1]
        a = random.randint(2, n - 1)

        # Compute Jacobi symbol
        jacobi = jacobi_symbol(a, n)

        # If Jacobi symbol is 0, n is composite
        if jacobi == 0:
            return False

        # Compute a^((n-1)/2) mod n
        exponent = (n - 1) // 2
        mod_val = mod_pow(a, exponent, n)

        # Adjust mod_val to be in range [-1, 0, 1] for comparison
        if mod_val == n - 1:
            mod_val = -1
        elif mod_val > 1:
            mod_val = 1

        # Check Euler criterion: a^((n-1)/2) ≡ (a/n) (mod n)
        if mod_val != jacobi:
            return False

    # Passed all tests
    return True
```

Рис. 2.4: Тест Соловэя-Штрассена

2.5 Тест Миллера-Рабина

Тест Миллера-Рабина является наиболее эффективным вероятностным тестом простоты. Он основан на том, что для простого числа p при представлении $p - 1 = 2^s \cdot d$ (где d нечетно) для любого a выполняется либо $a^d \equiv 1 \pmod{p}$, либо $a^{2^r d} \equiv -1 \pmod{p}$ для некоторого $0 \leq r < s$.

2.5.1 Код реализации

```
def miller_rabin_test(n, k=5):
    """
    Miller-Rabin primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Write n-1 as d*2^s with d odd
    s = 0
    d = n - 1
    while d % 2 == 0:
        d = d // 2
        s += 1

    # Perform k rounds of testing
    for _ in range(k):
        # Choose random a in [2, n-2]
        a = random.randint(2, n - 2)

        # Compute x = a^d mod n
        x = mod_pow(a, d, n)

        # If x == 1 or x == n-1, this round passes
        if x == 1 or x == n - 1:
            continue
```

Рис. 2.5: Тест Миллера-Рабина (часть 1)

```
        # Otherwise, square x repeatedly
        composite = True
        for _ in range(s - 1):
            x = mod_pow(x, 2, n)

            # If we hit n-1, we can stop this round
            if x == n - 1:
                composite = False
                break

            # If we hit 1, n is composite
            if x == 1:
                return False

        # If we never hit n-1, n is composite
        if composite:
            return False

    # Passed all tests
    return True
```

Рис. 2.6: Тест Миллера-Рабина (часть 2)

3 Тестирование алгоритмов

Для проверки работы алгоритмов была создана функция `test_all_algorithms`, которая тестирует все три алгоритма на наборе чисел, включая простые числа, составные числа и числа Кармайкла.

3.0.1 Результаты тестирования

```
======================================================================
LABORATORY WORK No. 5
Testing Probabilistic Primality Algorithms
======================================================================

Number     Expected   Fermat     Solovay-Strassen   Miller-Rabin Result
--------------------------------------------------------------------------------
2          Prime      Prime      Prime              Prime        ✓
3          Prime      Prime      Prime              Prime        ✓
4          Composite  Composite  Composite          Composite    ✓
5          Prime      Prime      Prime              Prime        ✓
7          Prime      Prime      Prime              Prime        ✓
9          Composite  Composite  Composite          Composite    ✓
11         Prime      Prime      Prime              Prime        ✓
13         Prime      Prime      Prime              Prime        ✓
15         Composite  Composite  Composite          Composite    ✓
17         Prime      Prime      Prime              Prime        ✓
19         Prime      Prime      Prime              Prime        ✓
21         Composite  Composite  Prime              Composite    ✓
23         Prime      Prime      Prime              Prime        ✓
29         Prime      Prime      Prime              Prime        ✓
561        Composite  Composite  Composite          Composite    ✓
1105       Composite  Composite  Composite          Composite    ✓
1729       Composite  Composite  Composite          Composite    ✓
2465       Composite  Prime      Composite          Composite    ✗
2821       Composite  Composite  Composite          Composite    ✓
6601       Composite  Prime      Composite          Composite    ✗
8911       Composite  Prime      Composite          Composite    ✗
999983     Prime      Prime      Prime              Prime        ✓
======================================================================
Note: Fermat test may fail on Carmichael numbers
Solovay-Strassen and Miller-Rabin are more reliable
======================================================================

======================================================================
Program finished
```

Рис. 3.1: Результаты тестирования

Из результатов видно, что: - Тест Ферма может ошибаться на числах Кармайкла (например, 2465, 6601, 8911) - Тесты Соловэя-Штрассена и Миллера-Рабина правильно определяют все

числа - Все три алгоритма правильно работают на малых числах

4 Вывод

В ходе лабораторной работы были реализованы три вероятностных алгоритма проверки чисел на простоту: тест Ферма, тест Соловэя-Штрассена и тест Миллера-Рабина. Экспериментально подтверждено, что тест Ферма может давать ложноположительные результаты на числах Кармайкла, в то время как тесты Соловэя-Штрассена и Миллера-Рабина являются более надежными. Точность всех тестов повышается с увеличением количества раундов тестирования.